

.gitignore mastery cheatsheet

Write precise patterns, choose the right scope, and keep generated, local, and sensitive files out of normal Git tracking.



- Patterns are not regex
- Affects untracked paths
- Order can change the result

MENTAL MODEL

one decision per path

Git selects the highest-precedence ignore source, then the last matching rule at that level decides. A match prevents an untracked path from appearing in normal add/status flows. It does not delete the path and does not untrack anything already in the index.

1 Put the rule in the correct scope

Team-shared

COMMIT

`.gitignore`
Versioned rules every clone should receive. May exist in the root or any subdirectory; patterns are relative to that file.

This clone only

LOCAL

`.git/info/exclude`
Personal repository rules that must not be committed, such as private scratch files or one-off tooling output.

All repositories

GLOBAL

`~/.config/git/ignore`
Machine-wide editor and OS noise. Git also accepts a custom path through `core.excludesFile`.

Precedence - highest to lowest

CLI patterns

>

nearest .gitignore

>

parent .gitignore

>

info/exclude

>

global

Tie-break rule

Within one precedence level, the last matching pattern wins. Put exceptions after the broader ignore rule.

2 Pattern language at a glance

<code># note</code>	Comment. A blank line is only a visual separator.	<code>build/</code>	Directories named <code>build</code> at any depth; trailing slash means directory only.
<code>\#report</code>	Literal name beginning with <code>#</code> . Backslash escapes special characters.	<code>/build/</code>	Only the directory beside this <code>.gitignore</code> ; root-only when placed at repository root.
<code>*.log</code>	Any matching name at any depth below this ignore file. <code>*</code> never crosses <code>/</code> .	<code>docs/*.pdf</code>	PDFs directly inside <code>docs</code> ; the slash makes it relative to this ignore file.
<code>temp?</code>	<code>?</code> matches exactly one non-slash character.	<code>docs/**</code>	Everything beneath <code>docs</code> , through unlimited directory depth.
<code>file[0-9].txt</code>	Character class or range: one character from the brackets.	<code>a/**/b</code>	<code>b</code> under <code>a</code> with zero or more directories in between.
<code>cache</code>	File or directory named <code>cache</code> at any depth.	<code>!keep.log</code>	Negates an earlier match and re-includes the path, if its parent remains traversable.

Whitespace: trailing spaces are ignored unless escaped, for example `name\` . A terminal backslash creates an invalid pattern.

!

Negation needs a reachable parent

Git does not descend into an excluded directory just to search for later exceptions.

DOES NOT WORK

`logs/`
`!logs/keep.log`

The parent directory itself is excluded.

CORRECT PATTERN

`logs/`
`!logs/keep.log`

The directory stays visible; its contents are filtered.

.gitignore command workflows

Create and edit the file safely, repair already-tracked paths, verify every rule, and commit with confidence.



3 Create, replace, append, and commit

macOS / Linux / Git Bash

shell

```
# create if missing
touch .gitignore

# append two lines
printf "node_modules/\n.env\n" >> .gitignore

# replace with a complete block
cat <<'EOF' > .gitignore
node_modules/
.env
/dist/
EOF
```

> replaces the entire file

>> appends to the end

Windows PowerShell

pwsh

```
# create if missing
New-Item .gitignore -ItemType File -Force

# replace with several lines
@"node_modules/", ".env", "/dist/" |
Set-Content .gitignore

# append one line
"/coverage/" | Add-Content .gitignore
```

Set-Content replaces

Add-Content appends

Open and version the shared rules

code .gitignore

git add .gitignore
git commit -m "chore: add ignore rules"

4 The path is already tracked - remove only from the index

One file

git rm --cached -- path/to/file

One directory

git rm -r --cached -- path/to/folder

Force-add once

git add -f -- path/to/ignored-file

Working files stay on disk. --cached removes only index entries; -- safely separates options from path names.

Re-evaluate the entire repository

git rm -r --cached .
git add .
git status
git commit -m "chore: refresh ignored files"

Use only when you intend a full index refresh. Inspect git status before committing.

5 Debug the rule instead of guessing

git check-ignore -v -- path

Show the winning source, line number, pattern, and path.

git check-ignore -v --no-index -- path

Evaluate even when the path is already tracked.

git ls-files -o -i --exclude-standard

List ignored, untracked paths.

git ls-files -c -i --exclude-standard

Find tracked paths that now match ignore rules.

git config --show-origin --get core.excludesFile

Reveal the configured global ignore file and its origin.

Verbose output anatomy

.gitignore:18:*.log logs/app.log

source : line : pattern pathname

No output normally means no rule matched. Use git status --ignored for a repository-wide view.

Security boundary

Ignore rules are not secret management. If a credential was committed, rotate it first, remove it from the index, and clean repository history when required. The file only blocks future normal adds.

.gitignore patterns you can use

Start with the universal baseline, add only the stack sections you need, then tighten paths to match your repository layout.



7 Universal starter

```
# OS-generated files
.DS_Store
Thumbs.db
Desktop.ini

# Root editor settings
/.idea/
/.vscode/*
!/.vscode/extensions.json

# Local secrets and environment
.env
.env.*
!.env.example
*.pem
*.key

# Logs, runtime, temporary files
*.log
*.pid
*.tmp
*.swp
.cache/

# Root build and test output
/dist/
/build/
/out/
/coverage/
```

Adapt, do not blindly paste

Keep generated output and machine-local state out. Track reproducible project inputs. A committed secret still requires rotation and history cleanup.

8 Stack add-ons

Node / frontend

npm, pnpm, frameworks

```
node_modules/
.npm/
.pnpm-store/
.next/
.nuxt/
.svelte-kit/
.vite/
.turbo/
```

Python

env, bytecode, tests

```
__pycache__/*
*.py[cod]
.venv/
venv/
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/
*.egg-info/
```

Java / JVM

Gradle, Maven, IDE

```
*.class
.gradle/
target/
/bin/
*.iml
```

.NET / Rust / Go

compiled output

```
# .NET
.vs/
bin/
obj/

# Rust
/target/

# Go
/bin/
*.test
coverage.out
```

9 Precision recipes

Root directory only

```
/vendor/
```

Same directory name anywhere

```
cache/
```

Direct children only

```
docs/*.pdf
```

Everything at unlimited depth

```
artifacts/**
```

Keep a placeholder file

```
uploads/*
!uploads/.gitkeep
```

Allowlist a source tree

```
*
!.gitignore
!src/
!src/**
```

Note: `.gitkeep` is only a convention. Git tracks files, not empty directories.

10 Final review checklist

- Generated?** Ignore compiler output, caches, logs, and downloaded dependencies.
- Personal or machine-specific?** Prefer `info/exclude` or the global file.
- Needed to reproduce the project?** Track it. Application lockfiles are usually committed; follow ecosystem policy for libraries.
- Broad rule or exception?** Anchor the path, place `!` later, and keep every parent directory traversable.
- Verified?** Run `git check-ignore -v -- path` before assuming the pattern works.

Five mistakes to avoid

- Expecting ignore rules to affect paths already tracked.
- Treating patterns as regular expressions.
- Forgetting that `*` cannot cross a slash.
- Ignoring a whole parent directory before unignoring one child.
- Using broad entries such as `*.json` when one generated folder is the real target.